

Aula 7 — Ponteiros e o operador ponto . em C



Agora chegamos a uma parte muito importante do C: os ponteiros.

No começo, ponteiro parece aquele assunto que o C inventou só para complicar a vida de quem está aprendendo. Mas, quando entendemos a ideia de memória, ele começa a fazer bastante sentido.

Nesta aula vamos entender:

- O que é um endereço de memória;
- O que é um ponteiro;
- O operador &;
- O operador *;
- Como acessar o valor apontado;
- A diferença entre . e ->;
- Ponteiros com struct;
- Como passar dados para funções usando ponteiros;
- Um pequeno projeto para praticar.

1. Primeiro: onde ficam as variáveis?

Quando fazemos:

```
int idade = 18;
```

o computador precisa guardar o número 18 na memória.

Podemos imaginar a memória assim:

Endereço	Valor
1000	18
1004	...
1008	...

O número 1000 representa, de forma simplificada, o endereço onde determinado dado está armazenado.

Então temos duas coisas diferentes:

idade → valor armazenado: 18

endereço: 1000

A variável possui um valor, mas também ocupa um espaço na memória.

2. O operador &

Em C, usamos & para descobrir o endereço de memória de uma variável.

Exemplo:

```
#include <stdio.h>

int main() {

    int idade = 18;

    printf("Valor: %d\n", idade);
    printf("Endereco: %p\n", (void*)&idade);

    return 0;
}
```

Você poderá obter algo parecido com:

```
Valor: 18
Endereco: 0x7ffe1234
```

O endereço pode ser diferente cada vez que o programa é executado. Isso é normal.

Aqui:

```
idade
```

significa:

Quero o valor da variável.

Enquanto:

```
&idade
```

significa:

Quero o endereço onde essa variável está armazenada.

Uma forma simples de lembrar:

```
idade → valor
&idade → endereço
```



3. Então o que é um ponteiro?

Um ponteiro é uma variável capaz de armazenar um endereço de memória.

Por exemplo:

```
int idade = 18;  
  
int *ponteiro;
```

Aqui:

```
int *ponteiro;
```

estamos dizendo:

ponteiro será capaz de guardar o endereço de uma variável do tipo int.

Agora podemos fazer:

```
ponteiro = &idade;
```

Temos:

```
idade  
|  
| valor = 18  
▼  
memória  
|  
| endereço guardado pelo ponteiro
```

Ou de uma maneira mais simples:

```
idade = 18  
  
ponteiro = endereço de idade
```

4. O operador *

Aqui aparece outro símbolo importante:

*

Quando usamos * com um ponteiro, podemos acessar o valor que está naquele endereço.

Exemplo:

```
#include <stdio.h>  
  
int main() {
```



```
int idade = 18;

int *ponteiro = &idade;

printf("Idade: %d\n", idade);
printf("Idade pelo ponteiro: %d\n", *ponteiro);

return 0;
}
```

Resultado:

```
Idade: 18
Idade pelo ponteiro: 18
```

Porque:

ponteiro

guarda o endereço.

E:

*ponteiro

acessa o valor que está naquele endereço.

Então:

& → pega o endereço

* → acessa o valor através do endereço

Essa é uma das coisas mais importantes para guardar nesta aula.

5. Podemos alterar uma variável através do ponteiro

Agora fica mais interessante.

Observe:

```
#include <stdio.h>

int main() {

    int idade = 18;

    int *ponteiro = &idade;

    *ponteiro = 20;

    printf("Idade: %d\n", idade);

    return 0;
}
```



Resultado:

Idade: 20

Mas nós não fizemos:

idade = 20;

Fizemos:

*ponteiro = 20;

Como ponteiro aponta para idade, estamos alterando o valor de idade.

Podemos imaginar:

ponteiro
|
▼
idade
18

Depois:

*ponteiro = 20;

fica:

ponteiro
|
▼
idade
20

6. Uma comparação simples

Imagine que uma casa possui um endereço:

Rua X, número 25

A casa é o lugar onde está o objeto.

O endereço não é o objeto.

É apenas a informação que permite encontrá-lo.

Com ponteiros acontece algo parecido:

Variável → casa
Endereço → localização da casa
Ponteiro → guarda a localização



Então:

```
int idade = 18;
```

é a variável.

```
&idade
```

é o endereço.

```
int *ponteiro = &idade;
```

é um ponteiro guardando esse endereço.

E:

```
*ponteiro
```

é acessar o valor que está naquele endereço.

7. Agora vamos falar do operador ponto .

Como estamos estudando struct, o operador ponto é muito importante.

Lembra da nossa estrutura?

```
typedef struct {  
    char nome[50];  
    int idade;  
    float nota;  
} Aluno;
```

Podemos criar:

```
Aluno aluno;
```

Agora podemos acessar os campos usando .:

```
aluno.nome  
aluno.idade  
aluno.nota
```

Por exemplo:

```
aluno.idade = 18;
```

Significa:

Acesse o campo idade que pertence ao aluno.



8. . significa acessar um campo da struct

Imagine:

```
aluno
├── nome
├── idade
└── nota
```

Quando fazemos:

```
aluno.idade
```

estamos dizendo:

Dentro de aluno, quero acessar idade.

Da mesma forma:

```
aluno.nome
```

acessa nome.

E:

```
aluno.nota
```

acessa nota.

Portanto:

. → acessa um campo de uma struct

9. Mas e quando temos um ponteiro para uma struct?

Aqui começa a ligação entre os dois assuntos.

Imagine:

```
Aluno aluno;
```

Temos uma struct normal.

Podemos fazer:

```
aluno.idade = 18;
```



Mas agora imagine que temos um ponteiro:

```
Aluno *ponteiro = &aluno;
```

Agora ponteiro não é o aluno diretamente.

Ele guarda o endereço do aluno.

Então não podemos simplesmente fazer:

```
ponteiro.idade
```

Isso está errado.

Quando temos um ponteiro para uma struct, usamos:

```
->
```

10. O operador ->

Exemplo:

```
#include <stdio.h>

typedef struct {
    char nome[50];
    int idade;
    float nota;
} Aluno;

int main() {

    Aluno aluno;

    Aluno *ponteiro = &aluno;

    ponteiro->idade = 18;
    ponteiro->nota = 15.5;

    printf("Idade: %d\n", ponteiro->idade);
    printf("Nota: %.2f\n", ponteiro->nota);

    return 0;
}
```

Aqui:

```
ponteiro->idade
```

significa:

Vá até a struct apontada por ponteiro e acesse o campo idade.



11. Diferença entre . e ->

Essa diferença precisa ficar muito clara.

Quando temos a struct diretamente:

```
Aluno aluno;
```

usamos:

```
aluno.idade
```

Quando temos um ponteiro para a struct:

```
Aluno *ponteiro = &aluno;
```

usamos:

```
ponteiro->idade
```

Então:

Struct normal

↓

aluno.idade

↓

usa .

Ponteiro para struct

↓

ponteiro->idade

↓

usa ->

12. Mas -> é a mesma coisa que (*ponteiro).

Sim.

Esta:

```
ponteiro->idade
```

é uma forma mais simples de escrever:

```
(*ponteiro).idade
```

Vamos entender.

Primeiro:

```
*ponteiro
```



significa:

Acesse a struct para a qual o ponteiro está apontando.

Depois:

```
(*ponteiro).idade
```

significa:

Acesse o campo idade dessa struct.

Por isso existe:

```
ponteiro->idade
```

que torna o código muito mais fácil de escrever.

13. Ponteiros em funções

Agora vamos usar ponteiros em uma situação muito importante.

Imagine esta função:

```
void alterar(int numero) {  
    numero = 100;  
}
```

E:

```
int main() {  
    int numero = 10;  
    alterar(numero);  
    printf("%d\n", numero);  
    return 0;  
}
```

O resultado será:

10

Por quê?

Porque a função recebeu uma cópia do valor.



14. Usando ponteiro na função

Agora:

```
void alterar(int *numero) {  
    *numero = 100;  
}
```

E:

```
int main() {  
  
    int numero = 10;  
  
    alterar(&numero);  
  
    printf("%d\n", numero);  
  
    return 0;  
}
```

Agora:

```
100
```

Porque enviamos o endereço:

```
alterar(&numero);
```

E a função recebeu esse endereço:

```
int *numero
```

Então conseguiu alterar o valor original:

```
*numero = 100;
```

15. Ponteiro com struct

Agora vamos juntar tudo.

```
#include <stdio.h>  
  
typedef struct {  
    char nome[50];  
    int idade;  
    float nota;  
} Aluno;  
  
void aumentarNota(Aluno *aluno) {  
  
    aluno->nota = aluno->nota + 1;  
}  
  
int main() {
```



```

Aluno aluno;

printf("Nome: ");
scanf("%49s", aluno.nome);

printf("Idade: ");
scanf("%d", &aluno.idade);

printf("Nota: ");
scanf("%f", &aluno.nota);

printf("\nAntes: %.2f\n", aluno.nota);

aumentarNota(&aluno);

printf("Depois: %.2f\n", aluno.nota);

return 0;
}

```

Se a nota for:

14

depois da função:

15

Observe duas coisas:

```
aumentarNota(&aluno);
```

Passamos o endereço do aluno.

E dentro da função:

```
aluno->nota
```

acessamos a nota através do ponteiro.

16. . versus -> na prática

Olha esta comparação:

```

Aluno aluno;

aluno.nota = 15;

```

Aqui temos uma struct normal.

Agora:

```

Aluno *ptr = &aluno;

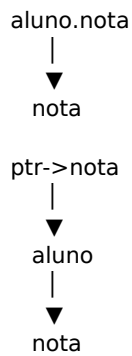
ptr->nota = 16;

```



Aqui temos um ponteiro.

Os dois estão trabalhando com o mesmo aluno.



17. Um pequeno projeto

Vamos fazer um sistema simples de aluno usando:

- struct
- typedef
- ponteiro
- &
- *
- .
- ->
- função
- if

```
#include <stdio.h>
```

```
typedef struct {
    char nome[50];
    int idade;
    float nota;
} Aluno;
```

```
void mostrarAluno(Aluno *aluno) {
```

```
    printf("\n=== DADOS DO ALUNO ===\n");
```

```
    printf("Nome: %s\n", aluno->nome);
    printf("Idade: %d\n", aluno->idade);
    printf("Nota: %.2f\n", aluno->nota);
```

```
    if (aluno->nota >= 10) {
        printf("Situacao: Aprovado\n");
    } else {
        printf("Situacao: Reprovado\n");
    }
}
```



```

}

void aumentarNota(Aluno *aluno) {

    aluno->nota = aluno->nota + 1;
}

int main() {

    Aluno aluno;

    printf("Nome: ");
    scanf("%49s", aluno.nome);

    printf("Idade: ");
    scanf("%d", &aluno.idade);

    printf("Nota: ");
    scanf("%f", &aluno.nota);

    mostrarAluno(&aluno);

    aumentarNota(&aluno);

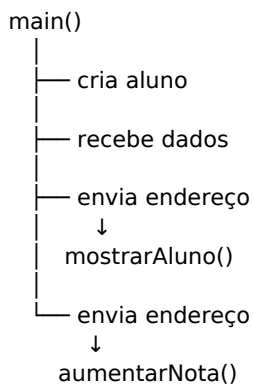
    printf("\n=== DEPOIS DO AUMENTO ===\n");

    mostrarAluno(&aluno);

    return 0;
}

```

Aqui já temos uma pequena arquitetura de programa:



18. O que você precisa memorizar desta aula

Não tente decorar 30 coisas. Comece por estas:

```
int idade = 18;
```

Variável normal.

```
&idade
```

Endereço da variável.



```
int *ptr = &idade;
```

Ponteiro guardando o endereço.

```
*ptr
```

Valor encontrado através do ponteiro.

Para struct:

```
aluno.idade
```

Use . quando você possui a struct diretamente.

```
ptr->idade
```

Use -> quando você possui um ponteiro para a struct.

A relação principal é:

& → pega endereço

* → acessa o valor através do endereço

. → acessa campo de uma struct

-> → acessa campo de uma struct através de ponteiro

Desafio da Aula 7

Crie uma estrutura:

```
typedef struct {  
    char nome[50];  
    float saldo;  
} Conta;
```

Depois:

1. Crie uma variável Conta.
2. Peça o nome e o saldo.
3. Crie uma função depositar().
4. Essa função deve receber um ponteiro para Conta.
5. Faça um depósito.
6. Mostre o novo saldo usando ->.

O objetivo não é copiar o projeto acima. Tente construir sozinho primeiro.



A ideia central desta aula é: ponteiro não é simplesmente “uma variável estranha com *”. Ele é uma variável que guarda um endereço, permitindo trabalhar diretamente com os dados armazenados naquele endereço.

